TRIP PLANNER PWA · FICHE MODULE · MEMBRE 3

Module 3 — Gestion des lieux et de l'itinéraire

Ce que fait le module, la logique métier qu'il porte, et ce que vous devez savoir présenter.

COLLECTION MONGODB	FONCTIONNALITÉ AVANCÉE	RÔLE TRANSVERSE
places	Carte interactive avec route quotidienne	—

1. Objectif du module

Construire l'itinéraire jour par jour : ajouter des lieux (recherche d'adresse ou clic sur la carte), les affecter à un jour, fixer heure, durée et coût estimé, les réordonner par glisser-déposer, et visualiser sur une carte le parcours de chaque journée avec les temps de trajet entre les étapes.

> Règle d'équipe : vous êtes le seul à écrire dans votre collection. Les autres modules passent par votre API (ou votre service), et toute modification d'un modèle partagé demande l'accord du groupe.

2. Modèle de données

Champ	Type	Règle / rôle
`trip`	ref trips	obligatoire
`name`	texte	1 à 120 caractères
`lat, lng`	nombres	−90..90 et −180..180
`address`	texte	≤ 300 caractères
`category`	enum	sight, museum, restaurant, cafe, hotel, nature, beach, shopping, nightlife, transport, other
`day`	entier | null	jour du voyage (à partir de 1) ; null = non planifié
`startTime`	HH:mm | null	heure de début
`durationMin`	entier	durée de la visite (0 à 1440)
`estimatedCost`	nombre	≥ 0, dans la devise du voyage
`order`	entier	position dans la journée
`notes`	texte	≤ 2000 caractères
`createdBy`	ref users	auteur

3. Logique métier

Création et modification

- Seuls les membres du voyage peuvent lire ou écrire (vérification via le voyage).
- À la création, le lieu est placé **à la fin de son jour** (ordre = dernier ordre du jour + 1).
- Si on change le jour sans préciser l'ordre, le lieu est ajouté **à la fin du nouveau jour**.

- Création idempotente avec un id client ; modification protégée par last-write-wins (409 si plus ancienne que le serveur).
- La suppression est idempotente : supprimer deux fois n'est pas une erreur, ce qui est utile pour le rejeu hors ligne.

Organisation de la journée

- `PUT /places/reorder` reçoit le jour et la nouvelle liste ordonnée ; il vérifie que tous les lieux appartiennent au voyage puis met à jour les ordres en une opération groupée.
- Dans l'interface, le glisser-déposer envoie une modification d'ordre par lieu déplacé : ce mécanisme fonctionne aussi hors ligne.
- Accessibilité : réordonnancement au clavier, et liste « Move to day » comme alternative au glisser-déposer.
- Liste paginée à **50** lieux, avec filtres par jour (ou non planifiés) et par catégorie, et recherche par nom.
- La durée estimée d'une journée = somme des durées de visite + trajets (le plus court entre marche et voiture).

4. Fonctionnalité avancée — Carte interactive avec route quotidienne

Carte Leaflet + tuiles OpenStreetMap. Marqueurs **numérotés et colorés par jour** (10 couleurs) ; les lieux non planifiés sont en gris.

Filtre par jour : on n'affiche que les marqueurs et la route du jour choisi. La carte se recadre automatiquement sur les lieux visibles.

Route du jour — `GET /trips/:id/route?day=N` : l'API appelle **OSRM** avec les étapes dans l'ordre et renvoie le tracé de la route, la distance et la durée en voiture de chaque tronçon.

Temps à pied estimé : distance à vol d'oiseau (formule de Haversine) × 1,25 à 4,5 km/h.

Mode dégradé : si OSRM ne répond pas (4 s), l'API calcule une estimation (distance × 1,3 à 40 km/h). Hors ligne, le navigateur fait le même calcul et affiche « estimate ».

Les routes sont mises en **cache 6 h** côté serveur ; les tuiles déjà vues restent disponibles hors ligne grâce au service worker.

Géocodage — `GET /geocode?q=` : recherche d'adresse via Nominatim, appelée par l'API (le client ne contacte jamais un service externe directement), avec cache et en-tête User-Agent. Sans réseau, on place le lieu en cliquant sur la carte.

5. API REST (préfixe `/api/v1`, réponse `{ data, message, errors }`)

Méthode	Route	Accès	Rôle
`GET / POST`	`/trips/:tripId/places`	membre	liste paginée (day, category, q) / créer
`GET / PATCH / DELETE`	`/trips/:tripId/places/:placeId`	membre	détail, modifier, supprimer
`PUT`	`/trips/:tripId/places/reorder`	membre	nouvel ordre d'une journée
`GET`	`/trips/:tripId/route?day=N`	membre	tracé + temps de trajet
`GET`	`/geocode?q=`	membre	recherche d'adresse

6. Écrans (front Next.js)

- Onglet « Itinerary » : carte, filtres par jour, journées avec glisser-déposer, temps de trajet entre les étapes
- Formulaire d'ajout et de modification avec recherche d'adresse et mini-carte cliquable

- Résumé jour par jour dans la vue d'ensemble ; carte de la page de partage

7. Fichiers dont vous êtes responsable

API — trip-planner-api

- `src/modules/places/ (place.model, place.routes, route.service)`

Front — trip-planner-web

- `src/app/trips/[id]/itinerary/page.tsx`
- `src/components/trips/PlaceForm.tsx`
- `src/components/map/TripMap.tsx`

8. Liens avec les autres modules

- **Module 2** : permissions, nombre de jours du voyage, file hors ligne.
- **Module 1** : l'export PDF et la page de partage lisent les lieux.
- **Module 6** : une entrée de journal peut être rattachée à un lieu ; le futur générateur IA créera de vrais lieux.

9. À démontrer à la soutenance

- ☐ Ajouter un lieu par recherche, puis un autre en cliquant sur la carte.
- ☐ Réordonner par glisser-déposer : la route et les temps se mettent à jour.
- ☐ Filtrer par jour ; déplacer un lieu vers un autre jour.
- ☐ Hors ligne : les temps deviennent des estimations, la carte déjà vue reste affichée.

10. Reste à faire

- Tests d'intégration des endpoints (création, réordonnancement, route).
- Requêtes Postman.
- Option : choisir le mode de transport (marche, voiture) pour la route.

11. Questions probables du jury

Pourquoi passer par l'API pour OSRM et Nominatim ?
Le cahier des charges impose que le client ne contacte aucun service externe directement ; cela permet aussi le cache et le respect des conditions d'usage.

Que se passe-t-il si OSRM est indisponible pendant la démo ?
L'API renvoie une estimation à vol d'oiseau et l'interface l'indique.